Hackathon Brief: Mapping the Short-Form Creator Universe

The Problem

Short-form video is huge, but there's no good map of what creators actually make content about. Platforms show broad categories and hashtags, but the real *niches* — where specific audiences gather around specific interests — are hidden unless you dig.

Your mission: build a system that **automatically produces a structured taxonomy of short-form content niches** from real platform data. Any creator or video should be classifiable into a specific niche.

Think of it as the Dewey Decimal System for TikTok / Reels / Shorts — but generated from data, not hand-curated.

Why This Is Hard

Anyone can list 20 obvious niches: fitness, beauty, gaming, food. The value is in the long tail — the ***micro-niches***. "Fitness" isn't a niche. "Calisthenics for tall guys over 30" is closer. "Hyrox prep" is closer still. A useful taxonomy has hundreds or thousands of leaf categories, structured as a tree, reflecting how creators actually cluster — not how marketers imagine they do.

You decide:

- Where the data comes from
- How to extract niche signals
- How to cluster signals into a tree
- How to name each niche so it's readable
- How to evaluate quality

No single right answer. That's the interesting part.

Deliverables

1. **Taxonomy file** — JSON/YAML tree. Each node: name, description, example hashtags/keywords/creators.
2. **Reproducible pipeline** — code that turns raw data into the taxonomy. One command, end-to-end.
3. **Classifier** — given a bio, caption, or hashtag list, return the most likely niche(s).
4. **Evaluation report** — coverage, granularity, quality (see below).
5. **Short writeup** — what you did, what worked, what you'd improve.

Approaches

Pick one, blend several, or invent your own.

A. Hashtag Co-Occurrence Graph

Scrape top hashtags on a platform. For each, sample posts and record co-occurring hashtags. You get a graph: nodes are hashtags, edges are co-occurrence counts.

- Run community detection (Louvain, Leiden, label propagation) to find clusters
- Each cluster = candidate niche
- Use an LLM (Gemini free tier or local Ollama) to name and describe each
- Vary resolution to get a hierarchy

Good: organic, finds emergent niches. **Hard:** noisy hashtags, junk clusters, naming is tricky.

B. Embedding-Based Clustering

Collect bios, captions, or post text. Embed with a sentence-transformer (`all-MiniLM-L6-v2` is free, runs locally). Cluster with HDBSCAN or k-means at different granularities.

- Sample documents per cluster
- LLM-name each one
- Build a hierarchy by clustering centroids

Good: semantic, ignores hashtag hygiene. **Hard:** short-text embeddings are noisy, clusters can be hard to interpret.

C. LLM-Driven Recursive Breakdown

Start with ~10 root categories. Ask an LLM to break each into sub-niches with examples. Recurse 3–4 levels deep. Validate against real data, fix gaps.

Good: fast, readable. **Hard:** LLMs invent plausible-but-fake niches, miss emergent ones, biased toward training data.

D. Hybrid

Use an existing taxonomy (IAB, YouTube categories, Reddit's subreddit graph) as a scaffold. Enrich with data-driven micro-niches from A or B. Often works best.

Mixing is encouraged.

Free Resources

Data sources

- TikTok hashtag pages — public, scrapeable with Playwright
- Instagram hashtag explore — public, rate-limited
- YouTube Data API v3 — 10,000 units/day free
- Reddit API — free tier, good for cross-checking niches

- Pushshift Reddit archives — free historical data
- Google Trends related queries — free
- Wikipedia category graph — free scaffolding
- Existing taxonomies — IAB Content Taxonomy 3.0, Curlie, Open Directory Project

Tools

- `sentence-transformers` — local embeddings, free
- `HDBSCAN`, `scikit-learn`, `networkx` — clustering and graphs
- `igraph` + Leiden — community detection
- Ollama + Llama/Mistral — local LLM, free, good for naming
- Gemini 2.0 Flash free tier — generous batch labeling
- DuckDB / SQLite — local storage

Infra

Run everything on a laptop. No cloud spend. If you want to pay for something, ask first.

Evaluation

1. **Coverage.** Classify a held-out sample of 1,000 creators/posts. What % land in a niche more specific than the top level? Target >85%.
2. **Granularity.** Plot leaf niche sizes. Good: power-law (few big, many small). Bad: everything in 5 buckets. Bad: 10,000 buckets of size 1.
3. **Stability.** Run the pipeline twice. Major structure should hold.
4. **Readability.** Can a non-expert read a niche name + description and understand who belongs there?
5. **Speed.** End-to-end runtime on a fresh dataset.
6. **Docs.** Can someone else run it without your help?

Stretch Goals

- **Cross-platform alignment** — are niches the same on TikTok, Reels, Shorts? Build a unified map.
- **Niche dynamics** — track how niches grow, shrink, or split over time.
- **Exemplar creators** — top 10 per niche.
- **Confidence scores** — return a distribution, not a single label. Creators often belong to multiple niches.
- **Open-set detection** — flag creators who fit no niche as candidates for new ones.

Timeline

- **Day 1:** Read this. Set up tools. Pick approach(es). Build data collection.
- **Day 2:** Generate v1 taxonomy. Build classifier. Start evaluation.
- **Day 3:** Iterate. Polish docs. Prepare demo + writeup.

What We Care About

Quality and rigor over flashy code. A boring pipeline that produces a sharp, well-named, well-distributed taxonomy beats a clever pipeline that produces mush.

When in doubt, look at the output and ask: *would this taxonomy actually help someone understand the creator landscape?* If not, iterate.

Ship something real.